

# Executable Archaeology of Early Bitcoin: Reconstructing the November 2008 Preview and Reviving 0.1.x IRC Peer Discovery

Joaquín Antonio De Vicente López

*bitcoin-history project*

June 24, 2026

**ABSTRACT**—In November 2008, months before the public launch, Satoshi Nakamoto circulated a preview of Bitcoin's source code that contained only four files (`main` and `node`) and therefore could not be compiled. We reconstruct it into a `wxWidgets` desktop node that compiles, links and executes on Windows XP—booting in a real window from which the user mines with a *Start mining* button—reproducing the network's earliest known genesis block *byte-for-byte*. In doing so we identify a concrete, independently verifiable consensus difference from the first public release (0.1.0): a renumbering of the script opcodes `OP_CODESEPARATOR` and `OP_CHECKSIG`, which is necessary for the genesis Merkle root to match. We also document a reproducible build toolchain and present runtime traces of the node creating its chain by mining and reloading it across restarts, and—for the first public releases in the same archive (0.1.0/0.1.3), recompiled from source with a small fix—show that their IRC-based peer discovery still works today, both on the surviving LFnet network and, after bridging four layers of protocol and anti-abuse drift, on modern Freenode.

**INDEX TERMS**—Bitcoin, software archaeology, genesis block, consensus serialization, reproducible builds, cryptocurrency history.

## I. INTRODUCTION

Bitcoin's official genesis block (January 2009) carries the headline *"The Times 03/Jan/2009 Chancellor on brink of second bailout for banks."* An earlier genesis exists, however, belonging to a private testnet that Satoshi abandoned; its hash is `000006b1...cfaa646`. It originates from the source preview Satoshi released on request in November 2008, preserved because the user *Cryddit* posted it on the BitcoinTalk forum in 2013 [1]; a narrative account of this alternative genesis block is given by *serhack* [2].

That preview is not self-contained: its own `readme` states *"These are just the main files. The rest is coming soon."* The support modules, the database layer and the program entry point are absent, and the code depends on `wxWidgets`, `Boost` and `Berkeley DB`. Consequently it has historically been an object of study rather than an executable artifact. This paper makes it run, and uses the byte-exact genesis block as an oracle to recover an undocumented detail of early Bitcoin consensus. It then turns to the first public releases held in the same archive—0.1.0 and 0.1.3—and shows that, recompiled from source with a one-line-class fix, their IRC peer discovery can be brought back to life on the surviving LFnet network and on modern Freenode (Section VI).

## II. THE RECONSTRUCTION

The objective was a native Windows XP binary derived from the preview—whose source, together with the first public releases (0.1.0/0.1.3) studied in Section VI, is archived by the Satoshi Nakamoto Institute [3]—while altering Satoshi's original code as little as possible. The components are:

- a) *Database layer*. The preview indexes transactions by on-disk position (a "TxPos" scheme), not the later CTxIndex of 0.1.0. Its interface, deduced from the call sites, was reimplemented over a flat-file key/value store in place of Berkeley DB.
- b) *Support modules*. Serialization, `uint256`, `bignum`, `script`, key handling and SHA were ported from `bitcoin-0.1.0`, the closest surviving source.
- c) *User interface and portability*. Because the preview's own `ui.cpp` was never released, a new `wxWidgets` front-end (`ui_app.cpp`) was written as a *recreation*—a small status window that reports the genesis hash, best block and height, peer count, and the wallet's keys and address, and mines from a *Start mining* button. It is not a recovery of Satoshi's never-shipped November 2008 UI; only the consensus and genesis are byte-exact. `Boost` and `Berkeley DB` were removed

(only `wxWidgets` is linked, per item a), and a partial Win32 ↔ POSIX shim was added for native verification.

d) *Unreleased subsystems*. Inert stubs stand in for the "market" subsystem (products, tables, atoms) that the preview references but whose code was never released.

e) *Period cryptography*. The build compiles against the OpenSSL 0.9.8 headers and links the `libeay32.dll` that Satoshi himself shipped, leaving `bignum.h` and `key.h` unmodified. Cross-compilation uses MinGW.

## III. THE BYTE-EXACT GENESIS

Once linked, the node executes the integrity self-test Satoshi left in the code. The computed Merkle root, however, did not match the hard-coded value: we obtained `97f26f69...` where `769a5e93...` was expected. Because the block structure was identical (same fields, identical coinbase `04695dbf0e`), the discrepancy had to lie in the *serialized bytes*.

### A. Anchoring the target

A message in an r/btc thread [4] supplied the genesis header preimage, which we verified to double-SHA256 to the genesis hash (Listing 1). This both confirms that the header serialization is correct and fixes the target: the coinbase transaction must serialize so that its double-SHA256 equals `769a5e93...`.

```
SHA256(SHA256(
  00..00                (hashPrevBlock = 0)
  E38380EC...935E9A76   (hashMerkleRoot, internal byte order)
  A00BC848              (nTime = 1221069728)
  14000000              (nBits = 20)
  BB290200 ))           (nNonce = 141755)
= 000006b15d1327...cfaa646 ✓ equals the genesis hash
```

Listing 1. Verified genesis block-header preimage.

### B. Resolving the difference

Dumping the coinbase bytes and comparing field by field, everything matched except the `scriptPubKey`. With the target hash known, the script's two opcode bytes (`OP-? <pubkey> OP-?`) were brute-forced, yielding a single solution: `0xA9` and `0xAA`. Our code, inherited from 0.1.0, used `0xAB/0xAC`.

The cause is an opcode renumbering. Between nov08 and 0.1.0, Satoshi *inserted* `OP_HASH160` (`0xA9`) and `OP_HASH256` (`0xAA`) into the opcode enumeration, shifting `OP_CODESEPARATOR` from `0xA9` to `0xAB` and `OP_CHECKSIG` from `0xAA` to `0xAC`. The genesis `scriptPubKey` is `OP_CODESEPARATOR <pubkey> OP_CHECKSIG`, so those two

bytes alter the transaction hash. Restoring the nov08 values makes both the Merkle root and the block hash exact (Listing 2).

```
hashMerkleRoot =
769a5e93fac273fd825da42d39ead975b5d712b2d50953f35a4fdebdec80
83e3 ✓
block hash =
000006b15d1327d67e971d1de9116bd60a3a01556c91b6ebaa416ebc0cfa
a646 ✓
```

Listing 2. Byte-exact genesis after restoring  
OP\_CODESEPARATOR=0xA9, OP\_CHECKSIG=0xAA.

### C. Provenance and validation

The recovery rests on a single external datum—the header preimage—posted as an anonymous, later-deleted comment in a public forum thread, whose authenticity was not independently established beforehand. The brute force closes the loop and validates it *a posteriori*: the preimage fixes the target Merkle root; the search over the  $2^{16}$  possible pairs of opcode bytes returns *exactly one* solution; and that solution yields a genesis whose Merkle root and block hash match the constants Satoshi hard-coded. A false positive—two incorrect bytes accidentally producing the target—has probability on the order of  $2^{-256}$  per trial, so the unique match is the genuine encoding rather than an artifact. The evidence is thus self-closing: it both authenticates the anonymous source and recovers the opcode values *without* access to nov08's unreleased `script.h`.

## IV. RUNTIME VERIFICATION

Under Wine—and, by construction, on Windows XP—the node boots into its wxWidgets status window, creates the genesis block, generates a real ECDSA key through `libeay32.dll`, and brings up the peer-to-peer subsystem on the original port 2222; the window reports the genesis hash, best block and height, the peer count, and the wallet's keys and address.

### A. Mining a chain

Pressing the window's *Start mining* button enables generation and launches `BitcoinMiner`, which mines blocks at the preview's test difficulty (`MINPROOFOfWork=20`). Listing 3 shows two consecutive blocks; note that the second block's `hashPrevBlock=000002` equals the prefix of the first block's hash, i.e. the blocks correctly chain back to the genesis (`hashPrevBlock=0000006`).

```
Mining enabled (-gen): launching BitcoinMiner...
ThreadSocketHandler / ThreadOpenConnections /
ThreadMessageHandler started
Running BitcoinMiner with 1 transactions in block
supercoin found
hash:
000002632794340f154c03ea4ebae3ac6de5e282f33071af98aa63a292ad
2313
target:
00000fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
ffff
CBlock(hashPrevBlock=000006, hashMerkleRoot=fb1c58,
nNonce=429672, vtx=1)
Running BitcoinMiner with 1 transactions in block
supercoin found
hash:
00000bf22eee364c15f08b05396a7676e91c1ce7ecf4d2e2f2390886cfdb
514a
target:
00000fffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
ffff
CBlock(hashPrevBlock=000002, hashMerkleRoot=e2528b,
nNonce=87838, vtx=1)
```

Listing 3. Live mining output (first two mined blocks; chained back to genesis).

### B. Persistence across restarts

After mining seven blocks the node is stopped and relaunched, this time *without* pressing *Start mining*. It rebuilds the block index from `blk0001.dat` by scanning the block file—

the preview keeps no persisted index—and reports a best height of seven (eight blocks including the genesis), confirming that the locally produced chain survives a restart (Listing 4).

```
Loading block index...
blocks=8 best_height=7
Loading wallet...
keys=1 transactions=0
Starting node (port 2222)...
bound to addrLocalHost = 192.168.1.67:2222
ThreadSocketHandler / ThreadOpenConnections /
ThreadMessageHandler started
```

Listing 4. Reload without mining: the chain persists at height 7.

The miner required one fix. The global `keyUser`—the key the coinbase pays to—was declared but never initialized, as the code that set it lived in the unreleased `init/UI`; `keyUser.GetPubKey()` therefore invoked `i2o_ECPublicKey()` on an `EC_KEY` with no public point and faulted. Initializing it at startup resolves the crash.

## V. LIMITATIONS

The node does not synchronize with today's Bitcoin network: it speaks the 2008 protocol at test difficulty, and the original testnet no longer exists, so the mined coins are of no value. The owner index and a few rarely-exercised `SIGHASH` branches—which lived in the unreleased code—are approximated, and the market subsystem is inert.

## VI. THE 0.1.x CLIENTS AND IRC PEER DISCOVERY

The same archive holds Satoshi's first public releases—0.1.0 (January 2009) and the near-identical 0.1.3. Run today, both sit at height 0 for two independent reasons. First, the miner refuses to run while isolated, blocking on `while (vNodes.empty())`; supplying a single peer (even an inert held TCP socket, which enters `vNodes` on `accept()` before any handshake) releases it, and with *Generate Coins* on the chain then grows on top of the January 2009 genesis at difficulty 1. Second, their only peer discovery is IRC—both are hard-coded to the `#bitcoin` channel on `chat.freenode.net`, where a node advertises itself with a nickname that base58-encodes its IP and port via a packed `struct ircaddr {int ip; short port;}` (`EncodeAddress`, `irc.cpp`); peers read the channel, decode the `u<base58check>` nicks back to addresses and connect. We revived this by **recompiling both GUI clients from source** with a small fix in `irc.cpp`, shipped as four ready-to-run, per-network builds under `irc/` (`freenode/` and `lfnet/`, each holding `bitcoin-0.1.0` and `bitcoin-0.1.3`) in [5].

### A. Rebuilding the GUI clients from source

Unlike the minimal November 2008 rebuild of Sections II–IV, which links only wxWidgets, 0.1.0/0.1.3 are full **wxWidgets** desktop applications backed by Berkeley DB, so the build needs a period GUI/database stack cross-compiled for 32-bit Windows. A single shared toolchain, under `toolchain/`, serves every from-source build in the project—the November 2008 rebuild, 0.1.0/0.1.3, and the IRC clients alike. `toolchain/build-toolchain.sh` cross-builds, into `toolchain/build/`, wxWidgets 2.8.12 (built `ANSI`, not `Unicode`, and `static`, to match the `wxmsw28` library names in Satoshi's `makefile`), Berkeley DB 4.8.30.NC (the `db_cxx` C++ API), Boost 1.55 (headers only—Bitcoin links no Boost libraries), and a `libeay32.a` import library generated—via `gendef/dlltool`—from Satoshi's own `libeay32.dll`; the period `OpenSSL 0.9.8` headers live in `toolchain/openssl/`. The cryptography is thus byte-for-byte Satoshi's, reused from the reconstruction.

Every project builds from its **original makefile, unedited**, through a helper `toolchain/make.sh` that adapts only the *environment*: a `PATH` shim so the `makefile`'s literal `g++/windres` resolve to the `i686-w64-mingw32` cross-tools, with `g++` driven in the period `-std=gnu++98`; and overrides of the `makefile`'s

INCLUDEPATHS/LIBPATHS/LIBS so they point at toolchain/ rather than the 2009 Windows install roots. Neither the makefile nor the source is ever touched. (The November 2008 rebuild's makefile, modelled on Satoshi's 0.1.0 one, instead asks for `-std=gnu++0x` and `-fpermissive`, because its reconstructed `serialize.h` uses `std::true_type` and its `main.cpp` takes the address of an `rvalue`.)

A modern GCC 10 / MinGW-w64 cross-compiler needs three source-level fixes to the *libraries* (never to Bitcoin's own source), packaged as reusable diffs in `patches/compilation/` and applied automatically by `build-toolchain.sh`: Berkeley DB 4.8 defines a function `__atomic_compare_exchange` that is now a GCC built-in (renamed in `dbinc/atomic.h`); `wxWidgets 2.8's` `filefn.cpp` fails to find `_mkdir/_rmdir` because its `wxPOSIX_IDENT` macro takes the underscore spellings for every compiler except `__MINGW64__`, whereas a 32-bit MinGW-w64 toolchain defines only `__MINGW32__` and those functions live in `<direct.h>` (added under an `#ifdef`); and the resource compiler `windres` must be passed only `-I/-D` flags, not the compiler flags from `wx-config`. A fourth issue—GCC's narrowing rejection of a case `-1` over a `DWORD`—needs no patch at all: building `wxWidgets` in `-std=gnu++98` sidesteps it. Finally, because Satoshi's makefile links the C/C++ runtime *dynamically*, the executable imports the MinGW runtime DLLs `libgcc_s_dw2-1.dll` and `libstdc++-6.dll` (the modern MinGW-w64 equivalents of the `mingwm10.dll` Satoshi shipped in 2009), which are placed next to `bitcoin.exe` and `libeay32.dll`—leaving the same dependency footprint (`libeay32.dll` plus XP system DLLs and these two runtimes) as the original 2009 binary.

The IRC clients are committed as four per-network folders—`irc/freenode/bitcoin-0.1.0`, `irc/freenode/bitcoin-0.1.3`, `irc/lfnet/bitcoin-0.1.0`, `irc/lfnet/bitcoin-0.1.3`—each being exactly `bitcoin-0.1.x/src` with the network's patch applied (`patches/code/<network>/<version>.patch`). That patch carries the `irc.cpp` fix (the `\r\n` line terminator and the `GetMyExternalIP` domain-name fix) and **bakes the network choice into the source** as a `#define` (Freenode: `#define IRC_FORCE_XNICK`, default server `chat.freenode.net`; LFnet: `#define IRC_SERVER "irc.1fnet.org"`), so each folder builds with no `-D` flags—the only difference between the Freenode and LFnet patch is that one line. More broadly, all patches are committed under `patches/`, deliberately split into `patches/code/` (version-to-version source diffs: `nov08-to-rebuild`, `nov08-to-0.1.0`, `0.1.0-to-0.1.3`, and the four IRC patches) and `patches/compilation/` (the modern-GCC library fixes above), so the whole pipeline is reproducible from the archived source plus committed diffs.

*Release vs. debug, and logging.* In `util.h`, `printf` is `#defined` to `OutputDebugStringF`, which opens a `debug.log` file only when `__WXDEBUG__` is set—i.e. in a *debug* build. Satoshi's distributed 0.1.x binaries were debug builds, which is why running them writes a `debug.log`. Our from-source binaries are *release* builds, necessarily, because they link the `wxWidgets` libraries built `--disable-debug` and a `__WXDEBUG__` application linked against *release* `wx` would mismatch its object layout; a release build therefore emits no `debug.log` and its trace goes to block-buffered `stdout`, invisible mid-run under Wine even though the client is fully registered—the reliable indicator being a *persistent* TCP connection to the `ircd` (an unregistered client is dropped within a minute). The static GUI links to ~10.5 MB; stripping the debugger symbol tables (`strip`) reduces each executable to ~6.8 MB, essentially the size of Satoshi's 6.4 MB original, without altering its behaviour.

*A startup hang, and its fix.* Before opening the network, `StartNode` calls `GetMyExternalIP`, which in the original code

dials hard-coded IP-echo addresses from 2008 (72.233.89.199 for *whatismyip.com* in 0.1.0; two more in 0.1.3). Those services are still online but moved off those IPs years ago, so each connection blocks in `SYN-SENT` until it times out—up to minutes under Wine, and twice in 0.1.3—before the IRC thread even starts. The window appears frozen ("Not Responding"); the correct response is to **wait**, not to force-quit, after which the node continues normally. Our `irc/` source removes the stall outright by resolving the services by **domain name** (`gethostbyname`) rather than the dead IPs, with a null guard; the rebuilt clients reach IRC within seconds.

#### B. LFnet still discovers peers—but they sit behind NAT

The IRC network the client moved to in later versions, **LFnet** (**irc.1fnet.org**), is still alive, still hosts `#bitcoin`, and—remarkably—still has real Bitcoin nodes advertising there. A one-line server change (`chat.freenode.net`→`irc.1fnet.org`, via an `IRC_SERVER` macro) is enough, because LFnet speaks the 2009 protocol unmodified and—unlike Freenode (Section VI-C)—does not ban the advertisement nick, so the original `u` prefix is kept here. The rebuilt client registers (001–004), JOINS, WHOs and decodes live `u<base58check>` nicks back into addresses—in our capture, 158.140.180.83:8333 and 76.17.176.13:8333 (Bitcoin Core's *BlueMatt* is still in the channel). One important caveat: *advertising* an address is not the same as being *reachable* at it—every node we found advertised on LFnet sat behind NAT and refused the connection (76.17.176.13 and 158.140.180.83 both did), so the discovery half works (the nick decodes to a real address) but the connect half fails unless the node is genuinely reachable (port-forwarded or on the same LAN). To close the loop for the mining demo we therefore advertised a connectable peer of our own (a bot in `#bitcoin` plus a listener); the client decoded it, connected, populated `vNodes`, and mined—the full 2009 path of discovery → connection → mining. No code change beyond the server name is required on LFnet; even the stock 2009 binary works there via a single `/etc/hosts` redirect.

#### C. Modern Freenode: four layers of drift

Freenode is reachable too, but only after peeling back four independent incompatibilities. (i) *Line endings*: the client terminates every IRC line with a bare `\r`, but Freenode now runs `InspIRCd-3`, which expects `\r\n` and otherwise never parses the client's NICK/PONG ("Registration timeout"). Ending the five lines the client sends in `\r\n`—the four format strings and the `'\r'` appended to the PONG—fixes it; this is the entire `irc.cpp` protocol fix, and it is harmless on LFnet. (Without recompiling, the same effect is obtained by a one-byte `0x0D`→`0x0A` patch of each terminator in the binary—Freenode also accepts a bare `\n`—or by a man-in-the-middle proxy.) (ii) *Hostname NOTICE*: before sending NICK the client blocks until a hostname NOTICE matches a literal substring; an IP with reverse DNS yields `Found your hostname`, which matches natively, so no action is needed in the common case. (iii) *IP reputation*: Freenode *Z-lines* addresses by reputation—our residential IP after repeated attempts, and, worse, any Cloudflare WARP egress, which is listed in the DroneBL proxy/VPN blocklist—so a clean IP is required (a fresh residential lease registered immediately: 001–004, JOIN, WHO, channel list, ChanServ welcome). (iv) *The advertisement nick is itself banned*: to advertise a peer one needs a bot whose nick is the `u<base58check>` of an address, but Freenode's anti-spam now rejects that very nick shape (432 ... Suspected spam), a relic of the 2009-era clients that once flooded it. The consequence reshapes the channel: of the 77 members our registered client enumerated in a WHO/NAMES of `#bitcoin`, **zero** decoded to a `u<base58check>` address and **all were people**—among them Adam Back, Luke Dashjr, Ruben Somsen and Lisa Neigut, and even the anti-spam entity whose name (Neo`Nemesis) appears in our own Z-lined errors. A

once machine-readable discovery channel is now simply a human chat room, its nodes evicted by the filter meant to stop their flood. We tested whether a different *prefix* would dodge it—a client advertising with `n<base58check>` rather than `u<base58check>`—in a clean side-by-side on one fresh IP: the `x<digits>` client registered, stayed connected and **mined four blocks** (height  $0 \rightarrow 4$ ) with no ban for the whole session, whereas on that *same* IP an `n<base58check>` nick registered (001–004), JOINed, and was **Z-lined about one second after the JOIN**. Same clean IP, opposite outcomes—so the trigger is the nick: the filter targets the base58-address *shape* (a letter plus a long mixed-case blob), not the `u` character, and only the addressless `x<digits>` form survives, which is why the Freenode build forces it with a source `#define IRC_FORCE_XNICK` baked into `irc.cpp` (no `-D` flag). One cannot advertise an address on Freenode at all; for the mining demonstration we therefore supply the peer directly—a held socket, or a proxy injecting a fabricated JOIN for a reachable address—after which the Freenode client connected to it and mined those blocks on the January 2009 genesis (height  $0 \rightarrow 4$ , proof-of-work found). The 2009 IRC bootstrap is therefore not dead, but the two networks fail in opposite ways: **Freenode is reachable again—you can register and mine, but not discover** (the channel is people, not nodes); **LFnet still does real discovery, but its advertised peers sit behind NAT** and refuse connection. Each works for exactly the half the other cannot. The Freenode half-failure is worth stating precisely, because it is a *policy* limit, not a protocol one: the client and the 2009 IRC bootstrap are fully capable of advertising and discovering on Freenode—the `\r\n` fix already gets a node registered, joined and reading the channel—and the *only* remaining blocker is the anti-spam rule that Z-lines the base58-address nick shape. Were that rule lifted (or were the same build pointed at any network that permits the advertisement nick—LFnet today, or a private/self-hosted IRC server one controls), the `u<base58check>` advertise-and-discover path would light up on it unchanged. This is the relevant fact for anyone wanting to **revive the legacy IRC bootstrap**: nothing in the client needs further repair; one only needs a host network whose policy allows the address nick.

## VII. FUTURE WORK

The wxWidgets GUI is no longer a pending item: the November 2008 rebuild already is a working windowed node that mines from a button (Sections II–IV). Three directions remain.

*a) Peer discovery for the November 2008 node.* The preview shipped *no* discovery whatsoever—no IRC, no seeds—so the rebuild can only mine standalone; Satoshi added the IRC bootstrap only for the public 0.1.0 release. Grafting that 0.1.0 IRC mechanism onto the nov08 core is the natural next step, and it immediately raises the question the revived 0.1.x clients already pose: *which* network, and whether one is enough. The architecture is pluggable—the server is a one-line macro—so three options present themselves.

*Community networks (e.g. Libera.Chat) are the wrong host.* Libera, where Freenode's community migrated after the 2021 takeover, would hit the same walls: to a social IRC network, thousands of nodes advertising with throwaway address-nicks and reconnecting is bot traffic, so its DNSBLs, rate-limits and acceptable-use policy would reject it exactly as Freenode does. There is also a hard protocol mismatch: the 2009 client speaks only **plaintext IRC on port 6667**, with no TLS and no SASL authentication, whereas modern networks increasingly expect a TLS connection (port 6697) and SASL login—so the old client cannot even present itself the way they want. The conflict is structural, not incidental.

*A dedicated ircd is the LFnet model, with its trade-offs.* Running a purpose-built server (which is what LFnet is) removes the anti-abuse fight entirely: the `u<base58check>` flood is expected traffic, not spam, and the operator sets NICKLEN, registration and line-ending policy to match the 2009 client. The cost is centralisation (a single seizable point, and a bootstrap that must itself be bootstrapped by a hard-coded address) and a trust problem: IRC authenticates nothing, so whoever controls the channel can advertise poisoned peers and *eclipse* new nodes—the very weakness, alongside the privacy leak of publishing one's IP in a public channel and poor scalability, that led Bitcoin to retire IRC around 2011 in favour of DNS seeds, hard-coded seeds and *addr gossip*.

*Federation and authentication restore the original spirit safely.* Rather than one network, the client could iterate a *list* of small ircds (0.1.3 already does this for its external-IP probe), so discovery survives any single network banning or vanishing—genuine censorship resistance, since silencing a node would require banning it everywhere at once. Layering a censorship-resistant transport (Tor/I2P) hides who is advertising and defeats IP-level blocking, and *signing* the advertised address with the node's key would close the eclipse/poisoning hole the 2009 design left open. Taken to its conclusion, the “IRC channel as a public bulletin board of signed node addresses” is essentially what a modern decentralised relay protocol such as Nostr already provides—making it a fitting 2026 successor to the mechanism reconstructed here.

*b) A native Linux (ELF) build.* The core already carries partial Win32  $\leftrightarrow$  POSIX shims (`compat.h`, and a `winsock2.h`  $\rightarrow$  BSD-sockets shim), so pure files compile to ELF (`sha.cpp` builds cleanly); completing the shim—thread synchronisation and `/dev/urandom` entropy in place of the remaining Win32 calls—would yield a native console node, and a wxGTK port would carry the GUI across. Cross-compiling to a Windows `.exe` run under Wine remains the supported path today.

*c) The fuller 0.1.0 UI on the nov08 core.* Optionally, the richer 0.1.0 desktop interface could be back-ported onto the November 2008 consensus core, replacing the minimal status window of the rebuild while keeping the byte-exact genesis underneath.

## VIII. CONCLUSION

The contributions are historical, educational and forensic rather than financial, and are two. First, the November 2008 preview is rendered executable and verifiable—as a working wxWidgets node that boots in its own window and mines from a button—reproducing the earliest known genesis block byte-for-byte and, through it, recovering an undocumented consensus detail—the renumbering of `OP_CODESEPARATOR` and `OP_CHECKSIG`—that checks against a value Satoshi himself hard-coded. Second, the first public releases (0.1.0/0.1.3) are recompiled from source with a small `irc.cpp` fix that revives their IRC peer discovery on the surviving LFnet network and, by bridging four layers of protocol and anti-abuse drift, on modern Freenode—carried in both cases through to mining on the January 2009 chain. Every from-source build—the rebuild, 0.1.0/0.1.3 and the four IRC clients—is driven from Satoshi's own *unedited* makefiles by one shared period toolchain (`toolchain/`: `wxWidgets 2.8.12 + Berkeley DB 4.8 + Boost 1.55 + OpenSSL 0.9.8` with Satoshi's original `libeay32.dll`), with both the library and source diffs committed under patches/ so the whole pipeline is reproducible. This toolchain differs from Satoshi's 2009 one only in what surrounds his source: MinGW GCC  $\sim$ 3.4 native on Windows becomes MinGW-w64 GCC 10 cross-compiling from Linux (still buildable natively on Windows from the same makefiles), and the `mingwm10.dll` runtime becomes `libgcc_s_dw2-1.dll + libstdc++-6.dll`, while the

OpenSSL 0.9.8 cryptography is identical. Because the compiler differs, a rebuilt executable is not byte-identical to Satoshi's, so his original 0.1.0/0.1.3 binaries are kept committed and a rebuild

only replaces them in the working tree. The project is archived at [5].

#### REFERENCES

- [1] Cryddit, "Bitcoin source (November 2008 preview)," BitcoinTalk forum, 2013. [bitcointalk.org/index.php?topic=382374.0](http://bitcointalk.org/index.php?topic=382374.0)
- [2] serhack, "The Story Behind the Alternative Genesis Block of Bitcoin." [serhack.me/articles/story-behind-alternative-genesis-block-bitcoin](http://serhack.me/articles/story-behind-alternative-genesis-block-bitcoin)
- [3] Satoshi Nakamoto Institute, "The Complete Satoshi—Source Code" (archive of the November 2008 preview and the 0.1.0/0.1.3 releases used here). [satoshi.nakamotoinstitute.org/code](http://satoshi.nakamotoinstitute.org/code)
- [4] "Has anyone tried to reverse the original pre-release Bitcoin...", r/btc, Reddit (thread containing the genesis header preimage in an anonymous, later-deleted comment). [reddit.com/r/btc/comments/7o0ejr](https://reddit.com/r/btc/comments/7o0ejr)
- [5] J. A. De Vicente López, *bitcoin-history* project (reconstruction and IRC-revived 0.1.x clients), GitHub, 2026. [github.com/Kino1994/bitcoin-history](https://github.com/Kino1994/bitcoin-history)